

MAGNA · SECURITY REVIEW

Magna Staking PR.

A security review of Staking PR.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Magna	Staking PR	Guardian	April 8 – April 15, 2026	April 22, 2026

§ 00

Contents.

36 pages — overview, confidence ranking, scope, 23 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 18 findings	P. 09
	Remediation Review · 5 findings	P. 29
§ 05	Disclaimer	P. 35
§ 06	About Guardian	P. 36

§ 01

Overview.

Guardian reviewed Staking PR across 2 review rounds. The final report includes 3 Low, 20 Informational findings.

AUDIT FIRM	Guardian	REPOSITORY	magna-eng/protocol-staking-evm
CLIENT	Magna	METHODOLOGY	Smart contract · Manual · Main Review · Remediation Review
PROJECT	Staking PR	MAIN REVIEW COMMIT	f46c88b4d6c4
PREPARED BY	Guardian	REMEDATION REVIEW COMMIT	04559b636e2b
AUDIT WINDOW	April 8 – April 15, 2026		
FINAL REPORT	April 22, 2026		

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	0	0	0	0	0
Low	3	0	0	0	3
Informational	20	0	7	1	12
All severities	23	0	7	1	15

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Staking PR in `magna-eng/protocol-staking-evm`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
L-01	Parameter Mismatch In Claim Event	Events	LOW	RESOLVED	p. 9
L-02	Multiple Config Values No Longer Observable	Configuration	LOW	RESOLVED	p. 10
L-03	Precision loss due to integer division	Math	LOW	RESOLVED	p. 11
I-01	Warning Regarding multiExecute	Warning	INFORMATIONAL	RESOLVED	p. 12
I-02	Typos	Typo	INFORMATIONAL	PARTIALLY RESOLVED	p. 13
I-03	Use Low Level Call Instead of Send	Best Practices	INFORMATIONAL	RESOLVED	p. 14
I-04	Multicall functionality offers limited benefits	Best Practices	INFORMATIONAL	RESOLVED	p. 15
I-05	Same Salt Reverts Identical Batch Deploys	DoS	INFORMATIONAL	RESOLVED	p. 16
I-06	Hook getters lack view	Informational	INFORMATIONAL	RESOLVED	p. 17
I-07	Missing events in state change functions	Events	INFORMATIONAL	ACKNOWLEDGED	p. 18
I-08	Penalty Hook / maxPenaltyPercentage Mismatch	Best Practices	INFORMATIONAL	RESOLVED	p. 19
I-09	Sensitive roles self-administered, no admin veto	Access Control	INFORMATIONAL	ACKNOWLEDGED	p. 20

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-10	Warning For Users Regarding Arbitrary Hooks	Warning	INFORMATIONAL	RESOLVED	p. 21
I-11	Post-withdrawal hook treats amounts as same kind	Compatibility	INFORMATIONAL	RESOLVED	p. 22
I-12	Hook invoked on zero-amount withdrawals	Validation	INFORMATIONAL	RESOLVED	p. 24
I-13	Nft multiplier can possibly be reused	Informational	INFORMATIONAL	RESOLVED	p. 25
I-14	Hook owners can DOS users	Trust Assumptions	INFORMATIONAL	RESOLVED	p. 26
I-15	Unnecessary pre-decrement	Gas Optimization	INFORMATIONAL	RESOLVED	p. 27

REMEDIATION REVIEW

I-01	Typos	Typo	INFORMATIONAL	ACKNOWLEDGED	p. 29
I-02	Contract comparison emits compiler warning	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 31
I-03	Hook trust comment understates upgrade risk	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 32
I-04	Unscoped salts blur deployment attribution	Trust Assumptions	INFORMATIONAL	ACKNOWLEDGED	p. 33
I-05	Missing CREATE2 address computation getter	Compatibility	INFORMATIONAL	ACKNOWLEDGED	p. 34

LOW **L-01**

RESOLVED

Parameter Mismatch In Claim Event

CATEGORY	Events
LOCATION	https://github.com/magna-eng/protocol-staking-evm/blob/f46c88b4d6c480dd97e6259e9d21da1ae283c85d/src/dynamicApy/interfaces/IDynamicStaking.sol#L128
REVIEW	Main Review

DESCRIPTION

`IDynamicStaking.Claim` names its third parameter `penaltyAmount`, implying that the value represents a penalty. However, when a user claims, the contract emits the actual reward amount: `emit Claim(sender, stakeIdPart, actualClaimedAmount, claimFee)`.

Off-chain indexers or dashboards that rely on ABI field names for semantics may misinterpret claimed rewards as penalties, leading to incorrect accounting or misleading alerts.

RECOMMENDATION

Rename the parameter and its associated comment in the interface (and any downstream consumers) to accurately reflect actual behavior.

RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

Multiple Config Values No Longer Observable

CATEGORY Configuration

LOCATION <https://github.com/magna-eng/protocol-staking-evm/blob/f46c88b4d6c480dd97e6259e9d21da1ae283c85d/src/dynamicApy/DynamicStaking.sol#L38-L64>

REVIEW Main Review

DESCRIPTION

Key parameters such as `minStakeAmount`, stake time bounds, withdrawal delays, penalty caps, the whitelist toggle, and the fee manager were changed from public to internal without introducing replacement getter functions. Additionally, the previous helper `getMinimumStakeAmount` was removed in this PR.

As a result, users and off-chain tools have no way to determine minimum stake amounts, lock periods, or whether proofless staking is allowed without directly inspecting raw storage.

RECOMMENDATION

Re-expose these parameters through public getters or dedicated view functions so that external systems retain visibility into the staking configuration.

RESOLUTION

Status: Resolved.

LOW **L-03**

RESOLVED

Precision loss due to integer division

CATEGORY Math

LOCATION <https://github.com/magna-eng/protocol-staking-evm/blob/f46c88b4d6c480dd97e6259e9d21da1ae283c85d/src/dynamicApy/hooks/multiplier/DefaultCompositeMultiplierHook.sol#L52-L84>

REVIEW Main Review

DESCRIPTION

`DefaultCompositeMultiplierHook.getMultiplier` composes four per-hook multipliers as $(A * B / \text{MULTIPLIER_SCALER}) * C / \text{MULTIPLIER_SCALER} * D / \text{MULTIPLIER_SCALER}$ with `MULTIPLIER_SCALER = 1e4`. Each `/ MULTIPLIER_SCALER` is integer division, so the result is floored three times in sequence. The mathematically exact value is $(A \times B \times C \times D) / \text{MULTIPLIER_SCALER}^3$ with a single rounding at the end; the current form drops fractional units after every intermediate step, so the composite multiplier is systematically $\leq$ the exact value. That biases virtual stake and reward share slightly down versus a single final division.

RECOMMENDATION

Compute in one shot after promoting to `uint256`, for example `uint256(A) * B * C * D / (uint256(MULTIPLIER_SCALER) ** 3)` then cast to `uint32` after a bounds check. With each hook capped at `5 * MULTIPLIER_SCALER` (`Common.sol`), the product fits in `uint256` without overflow.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

RESOLVED

Warning Regarding multiExecute

CATEGORY	Warning
LOCATION	https://github.com/magna-eng/protocol-staking-evm/blob/f46c88b4d6c480dd97e6259e9d21da1ae283c85d/src/dynamicApy/BatchDeploy.sol#L34-L43
REVIEW	Main Review

DESCRIPTION

The `multiExecute` function is public and performs external calls to arbitrary addresses with arbitrary calldata. As a result, users can execute a wide range of external calls through the MultiExecute contract.

This contract must never hold token balances and should not be granted approvals, as its entire token balance (excluding ETH) could be transferred and any approvals could be exploited via `multiExecute`.

RECOMMENDATION

Document this behavior and clearly warn users not to fund or approve the contract unless operations are executed atomically, as all funds held by the contract can be swept.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-02**

PARTIALLY RESOLVED

Typos

CATEGORY	Typo
LOCATION	https://github.com/magna-eng/protocol-staking-evm/blob/f46c88b4d6c480dd97e6259e9d21da1ae283c85d/src/dynamicApy/hooks/multiplier/NftMultiplierHook.sol#L11
REVIEW	Main Review

— DESCRIPTION

A state variable in `NftMultiplierHook` contains a typo, defined as `nftMulitplier` instead of `nftMultiplier`.

Additionally, the `UnexpetedEthReceived`, `AlreadyTermintated`, and `AlreadyForecefullyTermintated` errors contain typos.

— RECOMMENDATION

Update the variable names and errors with typos and all functions that reference them.

— RESOLUTION

Status: Partially resolved.

INFORMATIONAL I-03

RESOLVED

Use Low Level Call Instead of Send

CATEGORY Best Practices

LOCATION <https://github.com/magna-eng/protocol-staking-evm/blob/f46c88b4d6c480dd97e6259e9d21da1ae283c85d/src/dynamicApy/utils/WithDefundSupport.sol#L20>

REVIEW Main Review

DESCRIPTION

The `WithDefundSupport._defund` function uses `send` to transfer native ETH. It is best practice to use the low-level `call{value: amount}("")` pattern instead, as `send` forwards only 2,300 gas

RECOMMENDATION

Consider using low-level call instead of send.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-04

RESOLVED

Multicall functionality offers limited benefits

CATEGORY	Best Practices
LOCATION	BatchDeploy.sol
REVIEW	Main Review

DESCRIPTION

The `BatchDeploy` contract contains two functions - `deployContract()` and `multiExecute()` - and it inherits from the `Multicall` contract. The `Multicall.multicall()` function lets users batch different function calls of the same contract by utilizing `delegatecall`. In the case of the current contract, users can call `multiExecute()` and `deployContract()` in the same call. However, `multicall()` is non-payable, so it only works for `multiExecute()` calls that don't perform native token payments.

Functionally, there are no benefits of using `multicall`, because `multiExecute()` can be used to call `deployContract()` as well and there is no caller specific logic there. The only upside of using `multicall` is that it returns `bytes[]` return data, while `multiExecute()` ignores it.

RECOMMENDATION

Consider whether inheriting `Multicall` is necessary in this contract. If you decide to remove it, you can also modify `multiExecute()` to return the internal calls return data, just like currently `multicall()` does.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-05

RESOLVED

Same Salt Reverts Identical Batch Deploys

CATEGORY DoS

LOCATION <https://github.com/magna-eng/protocol-staking-evm/blob/f46c88b4d6c480dd97e6259e9d21da1ae283c85d/src/dynamicApy/BatchDeploy.sol#L29>

REVIEW Main Review

DESCRIPTION

`deployContract` loops over `deploymentCodes` and calls `Create2.deploy(0, salt, deploymentCodes[i])` with the same salt for every index. `CREATE2` resolves the contract address from deployer, salt, and the init code hash. If two entries share identical bytecode, they share the same init code hash, so the second deployment targets an address that already has code from the first iteration. OpenZeppelin's `Create2.deploy` then reverts. Because the whole call runs in one transaction, the entire batch reverts.

RECOMMENDATION

Derive a per-entry salt so identical bytecode still maps to distinct addresses

RESOLUTION

Status: Resolved.

INFORMATIONAL I-06

RESOLVED

Hook getters lack view

CATEGORY Informational

LOCATION <https://github.com/magna-eng/protocol-staking-evm/blob/f46c88b4d6c480dd97e6259e9d21da1ae283c85d/src/dynamicApy/interfaces/IMultiplierHook.sol>

REVIEW Main Review

DESCRIPTION

`IMultiplierHook.getMultiplier` and `IPenaltyHook.getPenaltyAmount` are declared as external with `returns (...)` but without the view modifier.

RECOMMENDATION

Add view to both interface signatures so implementations must be read-only at compile time.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-07**

ACKNOWLEDGED

Missing events in state change functions

CATEGORY	Events
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The following functions change critical protocol state without emitting any event: `modifyCampaign`, `setMerkleRoot`, `setEmergencyMode`, `setAllowStakingWithoutProof`, `setFeeParams`, `modifyMaximumRewardGenerationRate`, `modifyMinStakeAmount`, `withdrawAllPenaltyAmount`, `withdrawEthBalance`, `defundContractBalance` (DynamicStaking.sol); `setStakingContractAddress` (PostWithdrawalHookBase.sol); `setPenaltyRange` (StakeTimeRangePenaltyHook.sol); `setStakeAmountRange`, `setStakeTimeRange`, `setNftMultiplier` (DefaultCompositeMultiplierHook.sol).

RECOMMENDATION

Add events to all state-changing admin functions

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-08

RESOLVED

Penalty Hook / maxPenaltyPercentage Mismatch

CATEGORY Best Practices

LOCATION <https://github.com/magna-eng/protocol-staking-evm/blob/f46c88b4d6c480dd97e6259e9d21da1ae283c85d/src/dynamicApy/DynamicStaking.sol#L196>

REVIEW Main Review

DESCRIPTION

`DynamicStaking.unstake()` enforces two independent penalty checks:

1. `require(penaltyAmount == expectedPenaltyAmount)` user slippage protection.
2. `require(penaltyAmount * PENALTY_SCALER * 100 / actualUnstaked <= maxPenaltyPercentage)` immutable protocol cap set at deployment.

These serve different purposes: the first protects the user against a rogue admin front-running the penalty hook; the second is a hard on-chain guarantee to all stakers that penalties will never exceed a fixed percentage regardless of hook configuration.

However, `maxPenaltyPercentage` is `immutable` it cannot be changed after deployment. If it is set lower than the maximum penalty the hook can legitimately return (e.g. `maxPenaltyPercentage = 10%` but hook charges 50% for early exits), the second check will always revert for early unstakers. They cannot exit early under any circumstances and must wait until `unstakableFrom` when penalty is 0.

RECOMMENDATION

Document clearly that `maxPenaltyPercentage` must be $\geq$ the maximum penalty percentage the configured hook can ever return.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-09

ACKNOWLEDGED

Sensitive roles self-administered, no admin veto

CATEGORY Access Control
LOCATION DynamicStaking.sol
REVIEW Main Review

DESCRIPTION

`setRole()` calls `_setRoleAdmin(role, role)` for both `FORCE_LIQUIDATE_POSITION_ROLE` and `DEFUND_CONTRACT_BALANCE_ROLE`. Under OpenZeppelin AccessControl, only the role admin can grant or revoke that role, so `ADMIN_ROLE` cannot revoke a compromised holder. A compromised `DEFUND_CONTRACT_BALANCE_ROLE` holder can grant the role to additional addresses before anyone can react, then call `defundContractBalance()` and drain principal, rewards, penalties, and pending entries. There is no override path for `ADMIN_ROLE` once proliferation starts.

RECOMMENDATION

Use `_setRoleAdmin(role, ADMIN_ROLE)` for both roles so `ADMIN_ROLE` can revoke holders. For `DEFUND_CONTRACT_BALANCE_ROLE` especially, prefer a multi-sig rather than an EOA.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-10**

RESOLVED

Warning For Users Regarding Arbitrary Hooks

CATEGORY	Warning
LOCATION	https://github.com/magna-eng/protocol-staking-evm/blob/f46c88b4d6c480dd97e6259e9d21da1ae283c85d/src/dynamicApy/DynamicStaking.sol#L113-L114
REVIEW	Main Review

— DESCRIPTION

DynamicStaking accepts arbitrary multiplier, penalty, and post-withdrawal hook addresses in its constructor and does not enforce that these addresses point to the reference implementations provided in the repository. As a result, the deployer can supply any contracts they control.

A malicious or buggy hook can reject transactions, confiscate tokens, or reroute withdrawals. Users must perform due diligence and trust the pool deployer, as well as the specific hook contracts selected. There is no on-chain registry of audited hooks, nor any restrictions preventing the deployer from setting arbitrary hooks.

— RECOMMENDATION

Document this behavior clearly and warn users that deployers can freely choose hook contracts.

If only the reference hooks in this repository are intended to be used, maintain a list of approved hook contracts and enforce on-chain restrictions during deployment.

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-11

RESOLVED

Post-withdrawal hook treats amounts as same kind

CATEGORY Compatibility
LOCATION DynamicStaking.sol#L271-274
REVIEW Main Review

DESCRIPTION

A comment in `withdrawToRecipient` suggests that a custom post-withdrawal hook can be used to differentiate between claims and unstakes:

```
// withdrawToRecipient handles claims and stakes without any distinction, so if
// for example claims should always use direct transfer and stakes should use a hook
// then the solution is to disable direct transfer and create a special hook that can
// differentiate
// between claims and stakes
```

However, the hook interface does not provide enough information to achieve this. When a caller passes multiple entry types (e.g., `[Unstake, Claim]`), `removePendingEntries` processes both types sequentially and sums the results into a single `actuallyRemovedAmount`. The hook then receives:

- `amount`: the aggregate withdrawn amount across all types
- `pendingEntryTypes`: the array of types that were requested, not what was actually consumed

There is no per-type amount breakdown passed to the hook. Given `amount = 150` and `pendingEntryTypes = [Unstake, Claim]`, the hook cannot determine whether 100 came from unstakes and 50 from claims, or any other split.

A hook that needs to apply different logic to claims vs unstakes (e.g., different fee structures, routing, or access control) cannot function correctly for mixed withdrawals. The comment's suggested mitigation is incomplete — it relies on callers voluntarily making separate single-type calls, which cannot be enforced on-chain from the hook side.

RECOMMENDATION

If this is a desired feature, pass per-type amounts to the hook so it can apply type-specific logic without depending on caller cooperation. Otherwise, remove the comment.

I-11 Post-withdrawal hook treats amounts as same kind

CONTINUED

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-12

RESOLVED

Hook invoked on zero-amount withdrawals

CATEGORY Validation

LOCATION <https://github.com/magna-eng/protocol-staking-evm/blob/f46c88b4d6c480dd97e6259e9d21da1ae283c85d/src/dynamicApy/DynamicStaking.sol#L285-L287>

REVIEW Main Review

DESCRIPTION

In `DynamicStaking.withdrawToRecipient()`, when `isDirectTransfer = false` and `failOnZeroWithdrawal = false`, the `postWithdrawalHook.handlePostWithdrawal()` is called unconditionally; even when `actuallyRemovedAmount == 0` (for example no pending entries have matured yet).

```
if (actuallyRemovedAmount != 0) {
    token.safeTransfer(address(postWithdrawalHook), actuallyRemovedAmount);
}
// called regardless of actuallyRemovedAmount
postWithdrawalHook.handlePostWithdrawal(
    token, actuallyRemovedAmount, sender, recipient, stakeIdPart, pendingEntryTypes, extraData
);
```

Since integrators can deploy their own hook implementations extending `PostWithdrawalHookBase`, they must be aware that `_handlePostWithdrawal` can be invoked with `amount = 0`. A hook implementation that assumes a non-zero amount on every call will behave incorrectly in this case.

RECOMMENDATION

Add a prominent notice to `PostWithdrawalHookBase` and its documentation that `_handlePostWithdrawal` may be called with `amount = 0`. Integrators should always guard their logic with an early return or explicit check.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-13**

RESOLVED

Nft multiplier can possibly be reused

CATEGORY	Informational
LOCATION	NftMultiplierHook.sol#L37
REVIEW	Main Review

DESCRIPTION

The `NftMultiplierHook.getMultiplier()` function returns the stored `nftMulitplier` if the staker owns the NFT. Depending on which collection this hook is used for, users can transfer the same NFT among each other in order to benefit from the stake multiplier.

RECOMMENDATION

This behavior of the hook must be taken into consideration when configuring a multiplier and a collection.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-14**

RESOLVED

Hook owners can DOS users

CATEGORY	Trust Assumptions
LOCATION	hooks
REVIEW	Main Review

DESCRIPTION

There is a comment in `DynamicStaking` that highlights the possibility of a malicious hook owner DOS-ing user stakes by changing the multiplier.

```
// each staking/unstaking(with penalty) in which case the admin can DoS a particular user.  
// This is not a major issue as users can always submit his transaction to a node with private pool.  
// If DoS turns out to be an issue in the future, then an update window feature can be implemented.
```

The comment suggest that users can use private pools to avoid this, but in reality, the owner of the penalty hook can just set the penalty to an unreasonable value that the user wouldn't agree with, for example 100%, achieving the same DOS effect.

The owner of the `PostWithdrawalHookBase` can call `setStakingContractAddress()` and change the staking address to an invalid address - this would DOS withdrawals for all users.

RECOMMENDATION

Make sure the users are aware of the risks related to hook owners.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-15

RESOLVED

Unnecessary pre-decrement

CATEGORY Gas Optimization

LOCATION StakeTimeRangePenaltyHook.sol#L89
RangeMultiplierBase.sol#L20

REVIEW Main Review

DESCRIPTION

`StakeTimeRangePenaltyHook.getPenaltyPercentage()` and `RangeMultiplierBase.getRangeMultiplier()` perform redundant `--i` on the last line, as the `i` variable is no longer used after that.

```
function getPenaltyPercentage(uint32 value) internal view returns (uint32 penaltyPercentage)
{
    uint256 rangeLength = penaltyRange.length;
    uint256 i = 1;
    while (i < rangeLength && penaltyRange[i].remainingTimePercentageLowerBound <= value) {
        ++i;
    }
    penaltyPercentage = penaltyRange[--i].penaltyPercentage;
}

function getRangeMultiplier(MultiplierRangeEntry[] storage range, uint256 value)
internal
view
returns (uint32 multiplier)
{
    uint256 rangeLength = range.length;
    uint256 i = 1;
    while (i < rangeLength && range[i].lowerBound <= value) {
        ++i;
    }
    multiplier = range[--i].multiplier;
}
```

RECOMMENDATION

Consider using `i - 1` instead.

I-15 Unnecessary pre-decrement

CONTINUED

```
- penaltyPercentage = penaltyRange[--i].penaltyPercentage;  
+ penaltyPercentage = penaltyRange[i - 1].penaltyPercentage;  
  
- multiplier = range[--i].multiplier;  
+ multiplier = range[i - 1].multiplier;
```

— RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Typos

CATEGORY	Typo
LOCATION	Global
REVIEW	Remediation Review

— DESCRIPTION

Previously, there were 2 typos in the following errors: `NotYetForecefullyTerminTated` (forEcefully and terminTated) and `AlreadyForecefullyTerminTated` (forEcefully and terminTated). The latest commit fixed the `E` typo, but the `T` typo still remains.

Additionally, the following typos exist in comments, NatSpec documentation, and one named return identifier:

I-01 Typos

CONTINUED

- [`DynamicStaking.sol#L532`](<https://github.com/GuardianOrg/protocol-staking-evm-team1-1775663025703/blob/04559b636e2bf5ec1595557478bc2c4e702c67b9/src/dynamicApy/DynamicStaking.sol#L532>) — `rouge` should be `rogue` ("a small risk of a rouge admin").
- [`DynamicStaking.sol#L548`](<https://github.com/GuardianOrg/protocol-staking-evm-team1-1775663025703/blob/04559b636e2bf5ec1595557478bc2c4e702c67b9/src/dynamicApy/DynamicStaking.sol#L548>) — `transfering` should be `transferring`.
- [`IDynamicStaking.sol#L62`](<https://github.com/GuardianOrg/protocol-staking-evm-team1-1775663025703/blob/04559b636e2bf5ec1595557478bc2c4e702c67b9/src/dynamicApy/interfaces/IDynamicStaking.sol#L62>) — `migth` should be `might`, `vaule` should be `value`.
- [`IDynamicStaking.sol#L143`](<https://github.com/GuardianOrg/protocol-staking-evm-team1-1775663025703/blob/04559b636e2bf5ec1595557478bc2c4e702c67b9/src/dynamicApy/interfaces/IDynamicStaking.sol#L143>) — `migth` should be `might`, `vaule` should be `value`.
- [`FixedStaking.sol#L320`](<https://github.com/GuardianOrg/protocol-staking-evm-team1-1775663025703/blob/04559b636e2bf5ec1595557478bc2c4e702c67b9/src/fixdApy/FixedStaking.sol#L320>) — `exectute` should be `execute`.
- [`FixedStaking.sol#L352`](<https://github.com/GuardianOrg/protocol-staking-evm-team1-1775663025703/blob/04559b636e2bf5ec1595557478bc2c4e702c67b9/src/fixdApy/FixedStaking.sol#L352>) — `rouge` should be `rogue`.
- [`FixedStaking.sol#L460`](<https://github.com/GuardianOrg/protocol-staking-evm-team1-1775663025703/blob/04559b636e2bf5ec1595557478bc2c4e702c67b9/src/fixdApy/FixedStaking.sol#L460>) — `preceeding` should be `preceding`.
- [`IFixedStaking.sol#L96`](<https://github.com/GuardianOrg/protocol-staking-evm-team1-1775663025703/blob/04559b636e2bf5ec1595557478bc2c4e702c67b9/src/fixdApy/interfaces/IFixedStaking.sol#L96>) — named return `compoundingPeriodLenght` should be `compoundingPeriodLength` to match the function name.

— RECOMMENDATION

Fix all of the typos. The `compoundingPeriodLenght` named return in `IFixedStaking.sol` is purely cosmetic (named returns in interfaces are not part of the ABI selector), but should be aligned with the function name `compoundingPeriodLength` for consistency.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Contract comparison emits compiler warning

CATEGORY	Warning
LOCATION	src/dynamicApy/utils/WithDefundSupport.sol:16
REVIEW	Remediation Review

DESCRIPTION

In `WithDefundSupport._defund()`, the native-token branch is selected via `tokenParam == NATIVE_TOKEN`. Since both operands are contract-typed values, Solidity emits a compiler warning for direct contract comparison and recommends comparing their addresses explicitly. This does not currently change runtime behavior, but it adds avoidable warning noise to builds and makes the sentinel-address intent less explicit than an `address(...)` comparison.

RECOMMENDATION

Compare the sentinel values through `address(...)` in `WithDefundSupport._defund()`, for example `address(tokenParam) == address(NATIVE_TOKEN)`. This removes the compiler warning and makes the native-token check explicit and consistent with the surrounding zero-address validation.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

Hook trust comment understates upgrade risk

CATEGORY Documentation
LOCATION src/dynamicApy/DynamicStaking.sol:187
REVIEW Remediation Review

DESCRIPTION

The comment in `DynamicStaking.unstake()` says users only need to verify that the construction-time supplied penalty and multiplier hooks are not malicious as "a one-time check". That guidance is only accurate if the referenced hooks are immutable and cannot be upgraded or materially reconfigured after deployment. If a hook is upgradeable, owner-controlled, or depends on mutable external state, its behavior can change over time and the trust assumption must be revisited continuously. Leaving the comment as-is may cause integrators and users to underestimate the ongoing trust and monitoring requirements of the configured hooks.

RECOMMENDATION

Update the comment to clarify that hook verification is only a one-time check when the hooks are immutable and non-upgradeable. If hooks are upgradeable or admin-configurable, document that users and integrators should treat them as ongoing trust assumptions and monitor them accordingly.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-04

ACKNOWLEDGED

Unscoped salts blur deployment attribution

CATEGORY	Trust Assumptions
LOCATION	src/dynamicApy/MultiExecute.sol:33
REVIEW	Remediation Review

DESCRIPTION

`MultiExecute.deployContract()` derives each `CREATE2` salt as `bytes32(uint256(salt) + i)` without binding it to the caller. As a result, deterministic deployment addresses are namespaced only by the `MultiExecute` address, the derived salt, and the init code, not by the initiating user. Different users can therefore target the same deterministic deployment address by reusing the same `salt` and init code, or by choosing overlapping salt ranges across batches. This means callers cannot safely assume that a predictable address is uniquely tied to their own deployment flow or initiator identity. Any integrator or deployed contract logic that informally treats such an address as being attributable to a specific user, transaction origin, or deployment attempt is relying on a false trust assumption.

RECOMMENDATION

Scope the derived salt to the initiator in `deployContract()`, for example by hashing the caller together with the user-supplied salt and index. This makes deterministic deployment addresses user-specific and avoids collisions or attribution ambiguity across independent callers. If shared salts are intentional, document clearly that deterministic addresses produced by `deployContract()` are not bound to a particular caller.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05

ACKNOWLEDGED

Missing CREATE2 address computation getter

CATEGORY	Compatibility
LOCATION	src/dynamicApy/MultiExecute.sol:25
REVIEW	Remediation Review

DESCRIPTION

`MultiExecute` exposes `deployContract()` for deterministic `CREATE2` deployments but does not expose a view/helper function to compute the resulting deployment address from the supplied `salt`, batch index, and init code. While the address can be derived off-chain, the missing getter makes integration harder for contracts and on-chain workflows that need to reason about a future deployment address before it is created. This reduces composability, forces downstream users to reimplement the derivation logic externally, and increases the chance of mismatches if the derivation scheme is replicated incorrectly.

RECOMMENDATION

Add a view/helper function that returns the deployment address for a given `salt`, batch index, and deployment bytecode, and optionally a batch variant for multiple bytecodes. This would make deterministic deployments easier to consume from both off-chain tooling and on-chain integrations without requiring external reimplementation of the address derivation logic.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.